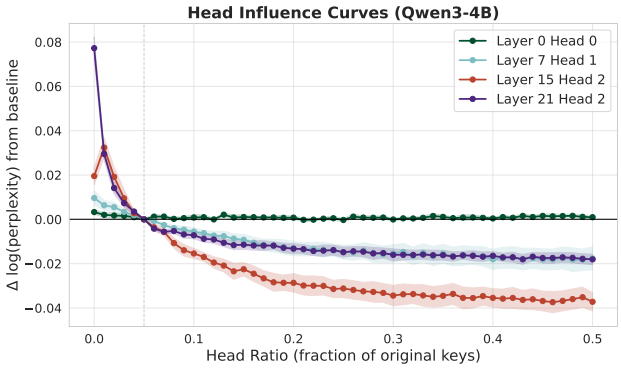

Figure 2. **Head sensitivity curves in Qwen3-4B.** We fix all KV heads to a baseline compaction ratio of $0.05\times$ and vary the compaction ratio of a single head. We report the change in loss relative to the baseline (lower is better) as a function of the varied head's compaction ratio. Curves are averaged over 10 QuALITY articles; shaded regions denote ± 1 standard error of the mean across articles. Some heads (e.g., `L0H0`) are largely insensitive to additional capacity, whereas others (e.g., `L15H2`) benefit substantially from storing more KV pairs.

separability across heads: we predict the effect of reallocating budget using single-head sensitivity curves measured with other heads held fixed.

The resulting schedule is an efficient model-specific nonuniform allocation procedure that prioritizes capacity for heads most sensitive to compaction. We found this allocation to be robust across instances/datasets, and thus this procedure only needs to be performed once for a given model. A visualization of the learned head budgets is provided in Appendix E.

3.5. Chunked Compaction

To support long contexts, we apply compaction independently to contiguous chunks of the input. Because Attention Matching can select and compact arbitrary token subsets, each chunk can be processed separately and the resulting KV caches concatenated to form a single compacted cache.

We consider two implementations. In *KV-based* chunking, we prefill the full context, slice out the KV states corresponding to each chunk, compact them independently, and merge the compacted chunks. In *text-based* chunking, we instead prefill and compact each chunk in isolation, and then align the resulting keys to their original global positions via a RoPE phase shift before merging. Detailed descriptions of both variants are given in Appendix C.3.

The text-based approach is an approximation because chunks are prefilled without cross-chunk interactions. In practice, we find that KV-based chunking more faithfully preserves model behavior, even when chunks are semantically independent, and therefore use it by default in all experiments, including baselines.

4. Results

We evaluate Attention Matching across a range of compaction strategies, measuring downstream quality and compaction efficiency on long-context benchmarks.

Compared Variants. We evaluate four representative compaction variants that trade off compaction time and downstream quality by varying how reference queries are obtained and how keys are selected. The variants are ordered by performance:

- **AM-OMP:** On-policy $\boldsymbol{Q}_{\text{ref}}$ from self-study + repeat-prefill; fit $(\boldsymbol{C}_k, \boldsymbol{\beta})$ jointly via OMP; fit $\boldsymbol{C}_v$ by least squares.
- **AM-OMP-fast:** Same as AM-OMP, but with OMP speedups ($k = 4$ keys selected per iteration, $\tau = 2$ iterations between NNLS refits; Appendix C.1).
- **AM-HighestAttnKeys:** On-policy $\boldsymbol{Q}_{\text{ref}}$ from self-study + repeat-prefill; select $\boldsymbol{C}_k$ by highest attention; fit $\boldsymbol{\beta}$ via NNLS; fit $\boldsymbol{C}_v$ by least squares.
- **AM-HighestAttnKeys-fast:** Same as AM-HighestAttnKeys, but with $\boldsymbol{Q}_{\text{ref}}$ coming only from repeat-prefill.

With self-study + repeat-prefill, we use at most $50{,}000$ reference queries per KV-head for each context/chunk. Importantly, this does not require a large number of generated tokens: under GQA, each token yields multiple query vectors per KV-head, and the majority of reference queries are obtained via repeat-prefill rather than self-study. On QuALITY, we generate on average approximately 5k self-study tokens and 7k repeat-prefill tokens per context.

Baselines. We compare against (i) **Cartridges** (Eyuboglu et al., 2025), which performs end-to-end gradient-based optimization of a compact latent KV cache for each context; (ii) baselines based on token pruning using attention scores (H2O (Zhang et al., 2023), SnapKV (Li et al., 2024), PyramidKV (Cai et al., 2025), KVzip (Kim et al., 2025b)), instantiated in our one-shot, question-agnostic setting as described in Appendix B.3; and (iii) **summarization**, which replaces the original context with a shorter textual summary, either produced for the full document or by summarizing chunks and concatenating the summaries (Table 5).

Evaluation protocol. We evaluate on QuALITY (Pang et al., 2022), a long-document comprehension benchmark (5–8k tokens; 15–20 questions per context), and LongHealth (Adams et al., 2024), a highly information-dense patient-records QA task (60k tokens per context; 100

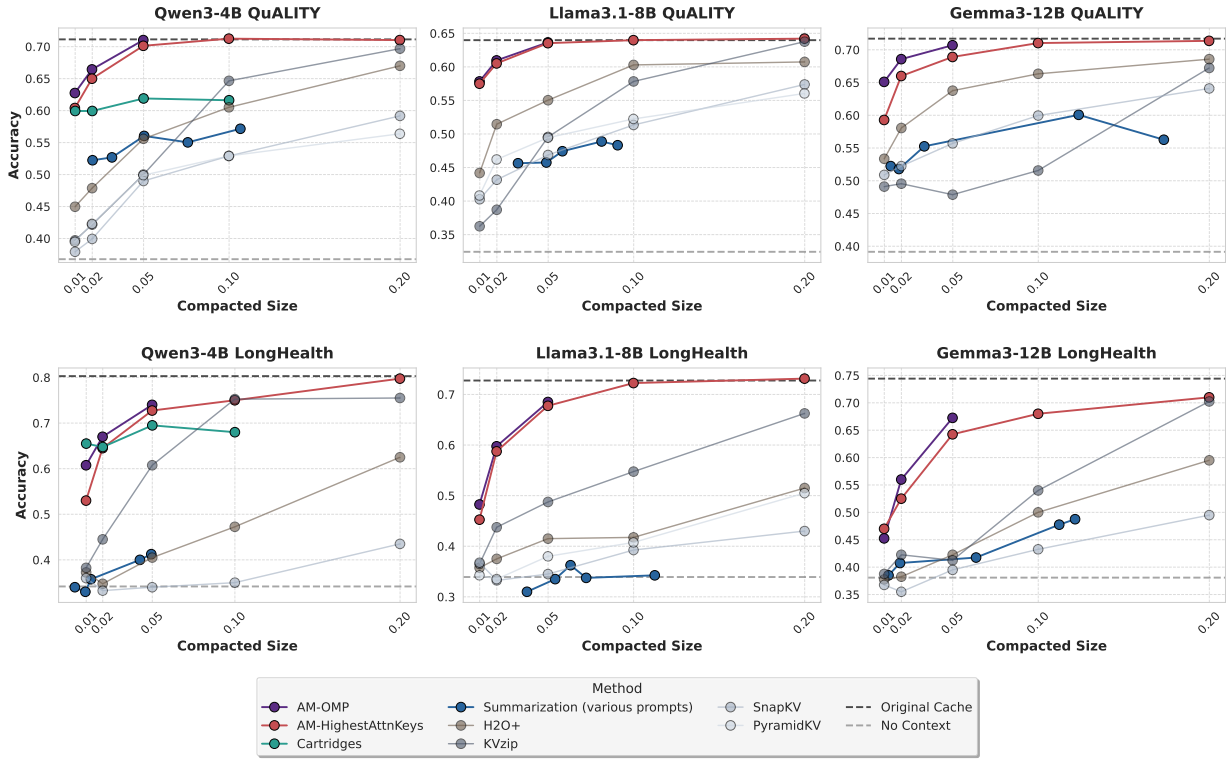

Figure 3. **Accuracy vs. compaction ratio across methods.** We compare AM-OMP and AM-HighestAttentionKeys against Cartridges, summarization, and four prior methods. Evaluations are conducted on QuALITY and LongHealth using Qwen3-4B, Llama3.1-8B, and Gemma3-12B. Attention Matching (AM) consistently outperforms other approaches across compaction ratios, while matching Cartridges' performance at ultra-high compaction.

questions per context, where each context aggregates 5 patients; 4 contexts total). See Appendix B.2 for a sample context and question.

For each context, we compact once and then answer all associated multiple-choice questions by decoding from the compacted cache. For all methods, we keep the chat template tokens fixed during compaction (e.g., BOS / system-prefix tokens, which can act as attention sinks (Xiao et al., 2024)) and re-attach them after compaction. We report the compacted size t/T with respect to the T article tokens, excluding chat-template tokens. While we found our method is robust to compacting these template tokens, we fix them for a fair comparison with baselines. We apply KV-based chunked compaction on LongHealth with 5 chunks for all methods except for Cartridges.

Our QuALITY evaluation is done over 50 contexts. Since evaluating Cartridges was computationally expensive, we only evaluate it on Qwen, on a fixed 20-context subset selected a priori; scores on this subset showed little deviation from the full set of 50 (Appendix B.2).

4.1. Main Results

We first analyze the trade-off between compaction time and downstream accuracy. Figure 1 plots this for Qwen3-4B on QuALITY at a fixed $50\times$ ratio. (See Figure 10 in the appendix for perplexity results.) Our Attention Matching (AM) methods trace the Pareto frontier and bridge the gap between fast heuristic baselines (e.g., summarization, H2O+, KVzip), which degrade significantly at this ratio, and optimization-heavy methods like Cartridges.

In Figure 3, we sweep the compaction ratio across 3 models on QuALITY and LongHealth. We observe that Attention Matching methods consistently outperform token-eviction baselines and summarization, particularly in the high-compaction regime ($20\times$–$100\times$). Performance degrades faster on LongHealth, which is highly information-dense (Appendix B.2). Summarization does particularly poorly on this dataset, matching the no-context baseline. KVzip sometimes matches Attention Matching performance, which we attribute to its non-uniform budget outperforming our precomputed budget at certain compaction ratios.

Stage	Method	Time (s)
Query generation	context-prefill	7
	repeat-prefill	8
	self-study	139
Key selection	Highest attention	3
	OMP	565
	OMP-fast (k=4, refit interval=2)	104
$\boldsymbol{\beta}$ fitting	NNLS	2.2
Value fitting	Least squares	1.8

Table 1. Wall-clock compaction time breakdown on a 60k-token LONGHEALTH context with Gemma-3-12B on a *single H200 GPU*.

4.2. Sliding Window Attention

The effectiveness of nonuniform compaction schedules gives some insight into why hybrid architectures with *sliding-window attention* (Child et al., 2019; Beltagy et al., 2020) and only a few global layers can retain much of the performance of full-attention models: even when trained with full attention, many KV-heads naturally specialize to modeling local context, while only a small number of globally attending heads capture the long-range dependencies.

Does this imply that compaction is less effective for these hybrid models? To verify that our improvements still hold on sliding-window models, we evaluate Gemma-3-12B, which has an aggressive 5:1 ratio of sliding-window layers to global layers. We compact only the global-attention layers, leaving the sliding-window layers unchanged. Accordingly, the reported compaction ratios are computed over only the global-attention portion of the KV cache. As shown in Figure 3, we require only slightly more conservative compaction ratios compared to the two full-attention models.

4.3. Compaction Efficiency

We profile the wall-clock compute cost of each component of our method on a 60k-token LONGHEALTH context using Gemma-3-12B (64 full-attention KV heads; head dimension d_{head}=256); see Table 1). All timings are measured on a single H200 GPU. We compute compacted keys and values in FP32 and then cast to BF16 for storage; we do not experiment with quantization in this work.

Query generation dominates runtime in this long-context setting and can likely be substantially optimized. All results here use a single GPU and self-study/repeat-prefill with chunked prefill with chunk size 4096.

Although we only evaluate models up to 12B parameters, we note that KV cache size varies quite little across different size variants of open-source models. For example, for the same sequence length, Qwen3-235B-A22B yields a KV cache only ∼30% larger than Qwen3-4B (96,256 vs. 73,728 elements per token). We therefore expect our compaction efficiency results to scale without much additional cost to much larger mixture of expert models commonly used in deployment.

Method	Cache Size (%)	Accuracy (%)
Full Context	100.00	71.5
Summarize	4.80	55.2
Summarize × (0.2× AM-OMP)	0.92	55.7
Summarize × (0.1× AM-OMP)	0.46	55.0
Summarize × (0.05× AM-OMP)	0.21	49.2

Table 2. Summarization plus Attention Matching. Applying AM-OMP on top of a summary enables up to ∼200× total compaction with accuracy comparable to summarization alone.

4.4. Extension: Summarization plus Attention Matching

While summarization rapidly degrades performance on tasks that require extracting knowledge and reasoning across long contexts (Section 4.1), it may be acceptable in cases where not all prior information needs to be retained (for example, when there exist retrieval or search mechanisms), or for filtering out noise or hallucinations.

In Table 2, we demonstrate that Attention Matching works on top of summarization: applying AM-OMP to summarized text achieves 200× compaction (6340→31 effective tokens on average) with performance comparable to summarization alone, which in our case provides only ∼ 20× compaction.

4.5. Ablations

We've introduced several concepts: post-compaction attention biases, fitted values, nonuniform head budgets, and query-sampling methods. In Figure 4, we leave out components of our main AM-OMP method and measure loss on reference generations. We see that every component is necessary to achieve the best results, with the least important being the use of self-study and the use of on-policy queries, and the most important being nonuniform head budgets.

We note that omitting attention biases and retaining the original values for selected keys still yields a reasonable approximation. However, refitting $\boldsymbol{\beta}$ and $\boldsymbol{C}_v$ via (nonnegative) least squares better aligns with our objective of approximating the original cache, adds little overhead, and consistently improves performance.

5. Related Work

While some prior works approach KV cache reduction as a per-token online problem—continuously evicting or merging tokens at each decoding step to maintain a fixed state size (e.g., Zhang et al., 2023; Oren et al., 2024)—we study compaction as a one-shot operation applied at the moment a context becomes too large. Methods in this setting are able to make globally informed decisions over the full prefix.

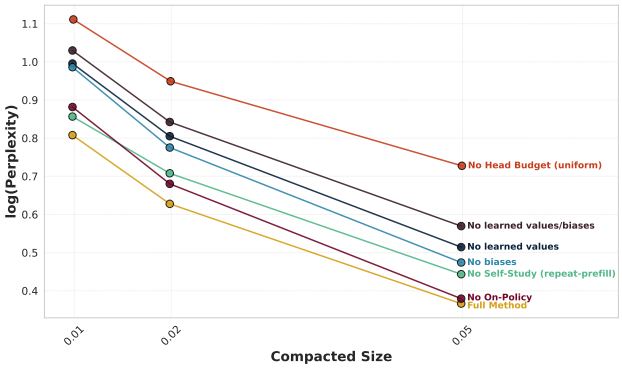

Figure 4. **Leave-one-out experiments.** We ablate our main AM-OMP method and measure average log-perplexity of generations decoded from the original cache, since this is lower in variance and well-correlated with downstream performance (Appendix F.2). In "no biases," we first compute OMP as usual and then zero out $\boldsymbol{\beta}$, keeping the keys selected by OMP.

Moreover, one-shot compaction can be applied repeatedly to maintain a fixed maximum state size, as in Appendix F.3.

Token-space approaches to handling long context generally involve combinations of retrieval-augmented generation (Lewis et al., 2020), context compaction via summarization or token dropping (Jiang et al., 2023; Li et al., 2023; Anthropic, 2025), and agentic memory management (Packer et al., 2024; Zhang et al., 2025). These approaches are largely orthogonal to latent-space compaction. While summarization rapidly degrades performance on tasks that require extracting knowledge and reasoning across long contexts (Section 4), it may be acceptable in cases where not all prior information needs to be retained (for instance, when there exist retrieval or search mechanisms). The two approaches can be combined with strong performance (Table 2); the most effective memory system may have a mix of retrieval and compaction in both token-space and latent-space.

In addition to the baselines that we evaluate (Zhang et al., 2023; Li et al., 2024; Cai et al., 2025; Kim et al., 2025b), a large body of other work explores token-merging (Wang et al., 2024; Zhang et al., 2024; Liu et al., 2025; Wan et al., 2025) and token-eviction (Oren et al., 2024; Ge et al., 2024; Tang et al., 2024; Chari & Durme, 2025; Łańcucki et al., 2025). The KVPress (Devoto et al., 2025) repository benchmarks over 20 such methods; we include the top-performing approach, KVzip (Kim et al., 2025b), among our baselines.

One line of prior work explores having models *generate* a set of soft tokens or "gist" tokens (Bulatov et al., 2022; Chevalier et al., 2023; Mu et al., 2023) that can be used to replace the original cache. These methods require training or finetuning the model to produce such representations, whereas our approach operates as a post-hoc procedure on any pretrained model without modification.

Our work is also related to Lexico (Kim et al., 2025a), which compresses keys and values by learning a *universal* dictionary per layer and uses orthogonal matching pursuit (OMP) at inference time to store each per-token key/value vector as a sparse code over the learned atoms. Lexico's objective is ℓ_2 reconstruction of KV vectors whereas we optimize compacted KV vectors for matching attention behavior.

DuoAttention (Xiao et al., 2025) proposes that attention heads be split into a small set of *retrieval* heads that use a full-length KV cache, and *streaming* heads that can run with a constant-length cache containing only attention sinks and recent tokens; they learn this partition via gradient optimization over per-head *gates* that blend full vs. streaming attention on synthetic data, then binarize the gates for deployment. Our nonuniform compaction shares the insight that heads vary in their sensitivity to KV capacity, but we learn continuous per-head budgets via *sensitivity curves* and compact all heads to varying degrees.

The notion of adding a per-token scalar attention bias was suggested in T5 (Raffel et al., 2020) and ALiBi (Press et al., 2022) as a form of positional encoding. Our method also employs attention biases, but for a different purpose: to correct for the attention mass received by the compacted cache.

6. Discussion

Disentangling logical KV length from physical cache size yields a practical primitive operation: KV compaction, which can be invoked on arbitrary portions of context whenever a sequence grows too large. Attention Matching is a natural paradigm for KV compaction because it approximately preserves the two ingredients that determine a block's contribution under concatenation—attention outputs and attention mass. Moreover, Attention Matching enables efficient algorithms for finding compact keys and values.

Our work still has several limitations. For one, while our approach is orders of magnitude faster than end-to-end gradient-based methods (i.e., Cartridges), our unoptimized OMP + self-study variants still required several minutes for compaction. And while our approach outperforms Cartridges in terms of accuracy at $50\times$ compaction rates, Cartridges outperforms our approach at more extreme ($100\times$) compaction rates on some benchmarks (LongHealth). We attribute this to the fact that gradient-based optimization can search over a wider space of compact representations—in particular, it is not restricted to optimizing intermediate attention outputs or selecting keys from the original cache—which becomes increasingly important as the compaction budget shrinks.